An Initial Evaluation of Exploratory Factor Analysis for Alpha Research

Manuel J. CEREZO, Abdoul B. ASSOUMANA, & Anthony COTTET

April 25, 2025

Profesor.Ying JIAO

Abstract

This project explores the application of Exploratory Factor Analysis (EFA) to build a statistical factor model for alpha-based investment strategies, using daily excess log-returns of 352 S&P 500 stocks from 2000 to 2025. We verified data suitability with Kaiser-Meyer-Olkin and Bartlett's tests, both confirming adequacy. After evaluating several heuristic criteria, we selected a 25-factor model estimated via Principal Axis Factoring (PAF), balancing parsimony and computational simplicity. The resulting model explains 51% of the total variance, meets essential statistical assumptions, and demonstrates strong fit; we conclude that the explained variance is sufficient given the covariance structure of the factor model and the stability of the idiosyncratic covariance matrix. A Varimax orthogonal rotation enabled economic interpretation, revealing clear sector groupings among stocks. Two alpha-based strategies derived from the model were evaluated: the *long-only* portfolio outperformed the benchmark in-sample (Sharpe Ratio = 0.69 vs. 0.25) and slightly out-of-sample (0.78 vs. 0.75), whereas the long-short portfolio underperformed. CAPM *beta* analysis suggested the presence of potential abnormal returns, warranting further investigation. The findings confirm the practicality of statistical factor models in quantitative portfolio management and invite us to continue researching the potential of dynamic factor models to develop implementable investment applications.

Contents

1 Introduction **4**

2 Mathematical Framework **5**
2.1 Notations 5
2.2 Asset Returns 5
2.2.1 Simple Returns 5
2.2.2 Excess returns & log-returns 5
2.3 The Factor Model 6
2.3.1 Identifiability and Rotational Indeterminacy 7
2.3.2 Selection Criteria for Number of Factors 8
2.3.3 Estimation Methods 8
2.3.4 Factor Rotation 10
2.3.5 Model Suitability 11
2.4 Application to Portfolio Management: Alpha Research 11
2.4.1 Introduction to Alpha 11
2.4.2 Alpha Estimation Method 12
2.4.3 Proposed Investment Strategy 12

3 Numerical Application **14**
3.1 Proposed Methodology 14
3.2 Data Collection and Pre-processing 14
3.2.1 Collecting, Processing and Assessing Excess Log-Returns 14
3.2.2 Suitability Assessment 16
3.2.3 Determining the Number of Latent Factors 16
3.3 Model Estimation: Principal Axis Factoring Method 17
3.3.1 Systematic Explained Variance 17
3.3.2 Assumptions & Estimation Performance 18
3.4 Factor Loadings Rotation 19
3.5 Latent Factor Interpretation 19
3.6 Alpha Estimation 20
3.7 Alpha Strategy Backtest 21

4 Discussion & Conclusion **24**

A Appendix A – Project Management & Organization **28**

B Appendix B – Python source code **30**

List of Figures

1 Enhanced EFA Methodology with Portfolio Management Application . . . 14
2 *U.S. Treasury 10 Year Bond Yields(%)* . . . 15
3 *Descriptive stats of returns (lhs), Covariance Matrix (rhs)* . . . 15
4 Distribution of the KMO Metric per Stock . . . 16
5 *Selecting Number of Factor: Scree Plot* . . . 18
6 Estimated Uniqueness/Ideosyncratic Covariance Matrix . . . 18
7 *Initial Factor Loadings (lhs) vs. Varimax Loadings (rhs) for All Stocks* . . . 19
8 Long-Short Portfolio vs S&P500, 2019-2025 (Indexed to 100) . . . 22
9 Long-Only Portfolio vs S&P500, 2019-2025 (Indexed to 100) . . . 23

List of Tables

1 *Differences Between PAF & PCA Methods* . . . 9
2 *Suitability Metrics for Factor Analysis* . . . 16
3 *Number of Factors Selected by Method* . . . 16
4 *Variance Explained by Extracted Factors* . . . 17
5 *Quality of Model Fit* . . . 19
6 *Sector Classification of Most Representative Stocks per Factor* . . . 20
7 *Top 10 Stocks by Estimated Alpha with Company Description* . . . 21
8 *Bottom 10 Stocks by Estimated Alpha with Company Description* . . . 21
9 *In-Sample and Out-of-Sample Performance Long-Short Portfolio vs Benchmark* . . . 22
10 *In-Sample and Out-of-Sample Performance Long-Only Portfolio vs Benchmark* . . . 23
11 *CAPM beta comparison.* . . . 25
12 *Betas of stocks comprising the long-only portfolio.* . . . 26

1 Introduction

The linear factor model has long been an essential tool for understanding financial asset returns. Its simplicity and versatility have ensured that it remains relevant even as far more complex and sophisticated models continue to emerge. At its core, the model relies on the idea that latent factors—unobserved but inferred statistically—can explain the observable correlations among asset returns. Factor models themselves predate their widespread use in financial economics; their statistical foundations extend back over a century, notably to Karl Pearson's pioneering work on principal component analysis (Pearson, 1901) and Charles Spearman's introduction of factor analysis (Spearman, 1904), which uncovered latent structures in psychometric data.

Finance scholars adopted these statistical methods with the development of modern portfolio theory and asset pricing models. The Capital Asset Pricing Model (CAPM), introduced by Sharpe (Sharpe, 1964) and Lintner (Lintner, 1965), proposed a single market factor driving returns but soon revealed empirical limitations. Ross's arbitrage pricing theory (Ross, 1976) offered a key breakthrough, generalizing the CAPM by statistically identifying multiple latent factors. This insight reflected the exploratory nature of statistical methods and underscored the importance of empirical investigation in financial modeling. Empirical studies, such as those by Fama and French (Fama and French, 1993), further validated a multifactor perspective by highlighting additional drivers of returns, such as firm size and valuation metrics.

Nevertheless, factor modeling presents persistent challenges. Estimating robust and stable covariance matrices remains difficult due to noisy financial data and high-dimensionality issues. Shrinkage estimation methods introduced by Ledoit and Wolf (2004) have significantly enhanced the reliability of these estimations and, in turn, practical portfolio construction. More recent work, including that of López de Prado (2022), employs causal graphs and causal discovery to bolster the robustness of factor analysis in portfolio management.

In this project, we aim to develop a simple, yet robust, statistical factor model of asset returns for the purpose of formulating an investment strategy based on the estimated model parameters. To do this, we employ the well-known statistical method of exploratory factor analysis. First, we present the model's mathematical and statistical underpinnings, emphasizing key theorems and definitions. Next, we discuss the concept of *alpha* in finance, necessary for comprehending the proposed investment strategy. Finally, we offer a preliminary estimation of the model, applying it to the returns of the constituents of the S&P500 market index.

2 Mathematical Framework

2.1 Notations

For the purpose of ease of reading and consistency, asset returns, in its many forms, will be called X_i, where i denotes the asset.

2.2 Asset Returns

2.2.1 Simple Returns

Asset returns are the building blocks of investment portfolios, and exhibit a wide range of stylized and empirical properties (Cont, 2001). They are derived from observed asset prices at specific time frames, ranging from seconds to months. For the purpose of this project, only daily asset prices and returns will be considered.

Definition 1 (Matrix of Asset Returns)**.** Let X_t^i be the return of asset i at time t, for $i = 1, ..., n$ (the number of available assets in the investment universe) and for $t = 1, ..., T$ (the number of periods). We call the *matrix of asset returns* $\mathbf{X} \in \mathbb{R}^{T \times n}$, the matrix where the i-th and t-th element is such that:

$$X_t^i = \frac{P_t^i - P_{t-1}^i}{P_{t-1}^i},$$

where P_t^i denotes the price and at period t.

The matrix of asset returns has been the subject of extensive research in financial econometrics and portfolio theory (see, e.g., Campbell, Lo, and MacKinlay, 1997), because it is the basis of all financial quantitative analysis.

Properties 1. Relevant properties for the formualtion and derivation of the statistical factor model include:

- The first and second moments of the return distribution are said to be *stationary* over the whole sample period, which is crucial for statistical inference in factor models.
- The return matrix $\mathbf{X}$ exhibits a structured dependence across assets, where common economic shocks, sectoral influences, and market-wide fluctuations create significant co-movements—justifying the introduction of factor models.
- The matrix $\mathbf{X}$ admits a well-defined covariance matrix $\Sigma_{\mathbf{x}}$, encapsulating both individual volatilities and cross-asset relationships.

2.2.2 Excess returns & log-returns

Beyond simple returns, two other commonly used return measures in financial modeling are *excess returns* and *log returns*. The notion of *excess returns* and *log returns*, were first introduced to address different modeling and empirical requirements. *Excess returns* by Sharpe (1964), Lintner (1965), and Mossin (1966), measure how an asset's return compares to a baseline risk-free rate. Meanwhile, *log returns* gained prominence in continuous-time asset pricing and volatility modeling—particularly following Merton (1973)—due to their mathematical tractability in capturing compounding effects over multiple periods.

Definition 2 (Excess & Log Returns)**.** Excess returns are defined relative to a benchmark rate (often the risk-free rate), capturing the additional compensation investors receive for taking on risk. Specifically, for asset i, the *excess return* at time t can be expressed as:

$$\tilde{X}_t^i \;=\; X_t^i - X_t^f,$$

where X_t^i denotes the asset's simple return and X_t^f the risk-free rate at time t.

Log returns are defined as:

$$x_t^i \;=\; \ln\Big(1 + X_t^i\Big) = \ln\Big(\frac{P_t^i}{P_{t-1}^i}\Big)$$

where X_t^i is the simple return of asset i.

Properties 2.

- Over shorter horizons or when returns remain moderate in magnitude ($X_t^i \approx 0$), x_t^i closely approximates X_t^i.
- Log returns enjoy certain mathematical conveniences, such as time-additivity over consecutive periods:

$$ln\Big(\frac{P_{t+p}}{P_t}\Big) = \sum_{i=1}^{p} x_{t+i}$$

Of course, we can also have excess log-returns, which often retain much of the empirical behavior observed in simple returns (e.g., volatility clustering, heavy tails), see Paleologo (2025), but can offer more tractable properties for certain statistical tests or modeling strategies. For the purpose of this project we will use excess log-returns of asset prices. And for simplicity we will continue to call the *matrix of excess log-returns* $\mathbf{X}$.

2.3 The Factor Model

The factor model is a statistical tool developed to simplify the complexity of large-dimensional data by uncovering a small set of latent variables—known as factors—that drive the co-movements among observed data. This sections develops the key underpinnings of the statistical factor model through the lens of Exploratory Factor Analysis.

Definition 3 (Factor Analysis Model)**.** Consider the vector $\mathbf{x_t} \in \mathbb{R}^{p\times 1}$ of observed asset returns with mean vector $\boldsymbol{\alpha}$ at time t. The classical factor analysis model decomposes this observed data into a combination of common latent factors and unique idiosyncratic components:

$$\mathbf{x_t} = \boldsymbol{\alpha} + \boldsymbol{\Lambda}\mathbf{f_t} + \boldsymbol{\epsilon_t}, \tag{1}$$

where $\boldsymbol{\Lambda} \in \mathbb{R}^{p\times m}$ is known as the loadings matrix and captures how strongly the asset returns correlated to underlying latent factors $\mathbf{f_t} \in \mathbb{R}^m$, with $m < p$ the number of latent factors. The error term $\boldsymbol{\epsilon} \sim N(0, \Psi)$ represents asset-specific variation not captured by the common factors, where $\boldsymbol{\Psi}$ is the the covariance matrix of the error term. We can, of course, generalize this to a time-series factor model such that:

$$\mathbf{X} = 1\boldsymbol{\alpha}^\intercal + \mathbf{F}\boldsymbol{\Lambda}^\intercal + \mathbf{E},$$

Property 1 (Covariance Structure)**.** The factor model imposes a structured covariance, separating common factor-driven variations from idiosyncratic risks:

$$\Sigma_{\mathbf{X}} = \Lambda\Phi\Lambda' + \Psi, \tag{2}$$

where typically $\Phi = I_m$ under the assumption that latent factors are standardized and uncorrelated, facilitating interpretability and simplifying the estimation process.

Proof.

$$\begin{aligned}
\Sigma_{\mathbf{X}} &= \mathbb{E}[\mathbf{X}\mathbf{X}'] \\
&= \mathbb{E}\big[(\Lambda\mathbf{F} + \boldsymbol{\epsilon})(\Lambda\mathbf{F} + \boldsymbol{\epsilon})'\big] \\
&= \Lambda\,\mathbb{E}[\mathbf{F}\mathbf{F}']\,\Lambda' + \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}'] + \Lambda\,\mathbb{E}[\mathbf{F}\boldsymbol{\epsilon}'] + \mathbb{E}[\boldsymbol{\epsilon}\mathbf{F}']\,\Lambda' \\
&= \Lambda\,\Phi\,\Lambda' + \Psi + 0 + 0 \\
&= \Lambda\,\Phi\,\Lambda' + \Psi.
\end{aligned}$$

□

Remark 1. The diagonal elements ψ_i of the uniqueness matrix Ψ quantify the specific variance of each asset not explained by common factors, often termed *idiosyncratic risk*, i.e the matrix Ψ is a diagonal covariance matrix, such as $\Psi = diag(\sigma_1, \ldots, \sigma_p)$, with σ_i the asset's specific volatility. Conversely, we often refer to the proportion of variance in asset returns accounted for by the latent factors as *commonalities*. This is a crucial aspect of the factor model for risk management and portfolio diversification.

2.3.1 Identifiability and Rotational Indeterminacy

Interestingly, the factor model is not uniquely defined, this is usually termed as rotational indeterminacy. Recall that an invertible matrix $\mathbf{T} \in \mathbb{R}^{m\times m}$ is said to be orthogonal, if and only if its columns are unit orthogonal vectors, such that:

$$\mathbf{T}'\mathbf{T} = \mathbf{T}\mathbf{T}' = I$$

Theorem 1 (Rotational Indeterminacy)**.** The factor model solution is identifiable only up to orthogonal or oblique rotation; formally, given an orthogonal matrix T, alternative equivalent solutions to the factor model are given by:

$$\Lambda^* = \Lambda T, \quad \Phi^* = T'\Phi T.$$

Proof. Recall the covariance structure of the factor model:

$$\Sigma_{\mathbf{X}} = \Lambda\Phi\Lambda' + \Psi.$$

Consider a rotated solution defined by $\Lambda^* = \Lambda T$ and $\Phi^* = T'\Phi T$, with T orthogonal. Then:

$$\begin{aligned}
\Sigma^*_{\mathbf{X}} &= \Lambda^*\Phi^*(\Lambda^*)' + \Psi \\
&= (\Lambda T)(T'\Phi T)(T'\Lambda') + \Psi \\
&= \Lambda(TT')\Phi(TT')\Lambda' + \Psi \\
&= \Lambda\Phi\Lambda' + \Psi \\
&= \Sigma_{\mathbf{X}}.
\end{aligned}$$

Thus, the rotated factor solution yields an identical covariance structure, demonstrating the rotational indeterminacy. □

Remark 2. This ambiguity is both a strength and a challenge: it allows for better interpretability of factor loadings, and allows for various practical applications, see Paleologo (2025).

2.3.2 Selection Criteria for Number of Factors

Selecting the correct number of latent factors m is a critical step in factor analysis. Several data-driven strategies have been proposed in the literature to guide this choice. Below, we summarize three widely used methods: threshold-based, maximum change points, and penalty-based criteria.

Definition 4 (Threshold-Based Method)**.** This method selects the number of factors by comparing the eigenvalues λ_i of the covariance or correlation matrix to a threshold derived from random matrix theory. For standardized data, we choose:

$$m = \max\left\{i \mid \lambda_i \geq 1 + \sqrt{\gamma}\right\},$$

where $\gamma = n/T$ is the aspect ratio of the matrix. A classical graphical variant is the *scree plot*, which involves visually identifying the last eigenvalue before a sharp drop in magnitude.

Definition 5 (Maximum Change Point Method)**.** This approach identifies the number of factors based on the largest gap between consecutive eigenvalues. Two variants are commonly used:

$$m = \underset{2 \leq k \leq k_{\max}}{\arg\max}\,(\lambda_{k-1} - \lambda_k), \quad \text{or} \quad m = \underset{2 \leq k \leq k_{\max}}{\arg\max}\,(\log \lambda_{k-1} - \log \lambda_k),$$

where $k_{\max}$ is a pre-defined upper bound, often selected iteratively, see Onatski (2005).

Definition 6 (Penalty-Based Method)**.** This method determines the number of factors by minimizing the trade-off between reconstruction error and model complexity. The objective is:

$$\min_{k \leq m \leq k_{\max}} \left\| \mathbf{X} - \hat{\mathbf{X}}^{(m)} \right\|^2 + k f(n, T),$$

where the penalty term is:

$$f(n, T) = \frac{n + T}{nT} \log\left(\frac{nT}{n + T}\right).$$

This balances goodness of fit with parsimony by penalizing overparameterization.

2.3.3 Estimation Methods

Estimating factor models requires careful statistical approaches to achieve accurate and robust identification of the latent structure underlying asset returns. In this project we will focus on two standard methods for factor extraction, namely principal axis factoring and maximum likelihood estimation, see Harman (1967).

i. Method n°1: Principal Axis Factoring

Definition 7 (Principal Axis Factoring (PAF))**.** PAF iteratively minimizes the discrepancy between the sample covariance $\mathbf{\Sigma}$ and the factor-structured covariance:

$$\min_{\Lambda,\Psi} ||\mathbf{\Sigma} - (\Lambda\Lambda' + \Psi)||^2, \tag{3}$$

subject to Ψ being diagonal and positive definite.

Remark 3. The problem does not admit a unique closed solution, because of the non-convexity and independance of estimated parameters. It has to be estimated iteratively through numerical algorithms.

Proposition 1. Following Harman, Bai and Li (2012) proposed the following PAF extraction method:

1. **Step 0.** Find the m largest roots of $\mathbf{\Sigma}$. Then compute

$$\widehat{\mathbf{\Sigma}} \;=\; (U\,L^{1/2})\,(U\,L^{1/2})' \;=\; \Lambda\,\Lambda'.$$

   where, the matrix U contain the eigenvectors while the diagonal elements of L contain the eigenvalues.

2. **Step 1.** Define

$$\mathbf{\Sigma}^* \;=\; \mathbf{\Sigma} \;-\; [\mathbf{I} \;-\; \mathrm{diag}(\widehat{\mathbf{\Sigma}})].$$

3. **Step 2.** Find the m largest roots of $\mathbf{\Sigma}^*$. Recalculate

$$\widehat{\mathbf{\Sigma}} \;=\; (U\,L^{1/2})\,(U\,L^{1/2})' \;=\; \Lambda\,\Lambda'.$$

   If $\widehat{\mathbf{\Sigma}}$ does not change from iteration to iteration, stop. Otherwise, return to Step 1.

Remark 4. It is important to highlight the differences between the PAF method and common dimensionality reduction methods such as Principal Component Analysis (PCA). The following table summarizes these key differences:

Aspect	**PAF**	**PCA**
Objective	Model $\Lambda\Lambda' + \Psi$	Decompose WW' (no Ψ)
Solution	Iterative	Closed-form (eigenvalues)
Constraints	Ψ diagonal	None
Interpretation	Probabilistic model	Descriptive method

Table 1: *Differences Between PAF & PCA Methods*

ii. Method n°2: Maximum Likelihood Estimation

The Maximum Likelihood method is a well-established and popular statistical procedure for estimating the unknown population parameters. It is nonetheless computationally demanding for large datasets, and it forces us to make important assumptions about the probability distribution of the asset returns.

Definition 8 (Maximum Likelihood Estimation (MLE))**.** Under the assumption of zero-mean, independent factors, and independence between the *commonalities* and *uniqueness* in the factor model, the matrix $\mathbf{X}$ follows a multivariate normal distribution with covariance matrix $\mathbf{\Sigma}$. The likelihood function for a sample of size n is:

$$\mathcal{L}(\Lambda, \Psi) = \frac{1}{(2\pi)^{np/2}|\Sigma|^{n/2}} \exp\left(-\frac{n}{2}\text{tr}(S\Sigma^{-1})\right),$$

$$\ln \mathcal{L}(\Lambda, \Psi) = \ell(\Lambda, \Psi) = -\frac{n}{2}\left[\ln|\Sigma| + \text{tr}(S\Sigma^{-1}) + p\ln(2\pi)\right],$$

$$\ell(\Lambda, \Psi) = -\frac{n}{2}\left[\ln|\Sigma| + \text{tr}(S\Sigma^{-1}) - \ln|S| - p\right],$$

By replacing $\mathbf{\Sigma} = \mathbf{\Lambda\Lambda'} + \mathbf{\Psi}$, we get the log-likelihood function to maximize:

$$\ell(\Lambda, \Psi) = -\frac{n}{2}\left[\ln|\mathbf{\Lambda\Lambda'} + \mathbf{\Psi}| + \text{tr}((\mathbf{\Lambda\Lambda'} + \mathbf{\Psi})^{-1}S) - \ln|S| - p\right], \tag{4}$$

where S is the sample covariance matrix.

Remark 5. Important remarks of the ML estimation of the factor model include:

- The log-likelihood function $\ell(\Lambda, \Psi)$ is non-convex.
- We typically use iterative optimization approaches (e.g., Newton-Raphson, Expectation-Maximization, gradient descent).
- Identifiability constraints must be imposed on Λ for uniqueness of the solution.
- We can evaluate the global adequacy of the factor model using the following test statistic where $d = \frac{p(p+1)}{2} - (pk + p)$.

$$T = (n-1)\left[\log|\hat{\Sigma}| + \text{tr}(\hat{\Sigma}^{-1}S) - \log|S| - p\right] \quad \text{, with} \quad T \sim \chi^2_d$$

- There are other asymptotic tests that allow the construction of confidence intervals, such as Wald, likelihood ratio, and Rao score tests.

Theorem 2 (Consistency and Normality of MLE)**.** MLE estimators $(\hat{\Lambda}, \hat{\Psi})$ are consistent and asymptotically normal under standard regularity conditions:

$$\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} N(0, \mathcal{I}^{-1}(\theta)),$$

where $\mathcal{I}(\theta)$ is the Fisher information.

2.3.4 Factor Rotation

As mentioned ealier, factor rotation is essential in exploratory factor analysis to improve interpretability without altering the model's fit. First introduced by Thurstone (1947), factor loadings rotations can achieve a simpler structure where each factor loads strongly on a subset of variables. For the purposes of this project, we will only focus on orthogonal rotations.

Definition 9 (Orthogonal Varimax Rotation)**.** The Varimax rotation maximizes the squared variance of each factor, achieving a "simpler" structure. The method consist on applying an orthogonal rotation that maximizes the Varimax Criterion $Q(\Lambda)$:

$$Q(\Lambda) = \frac{1}{p}\sum_{j=1}^{m}\left[\sum_{i=1}^{p}\lambda_{ij}^4 - \frac{1}{p}\left(\sum_{i=1}^{p}\lambda_{ij}^2\right)^2\right].$$

Property 2. Orthogonal rotations preserve factor independence, while oblique rotations permit factor correlations.

2.3.5 Model Suitability

Before proceeding with factor analysis, it is important to assess whether the model is appropriate for the data. In high-dimensional settings, two classical tests are commonly used to evaluate suitability: the Kaiser-Meyer-Olkin (KMO) measure of sampling adequacy and Bartlett's Test of Sphericity.

Definition 10 (Bartlett's Test of Sphericity)**.** This test evaluates whether the observed correlation matrix Σ significantly differs from the identity matrix. Formally:

$$\begin{cases} H_0 : \Sigma = I_p & \text{(no correlations among variables)} \\ H_a : \Sigma \neq I_p & \text{(sufficient correlations exist)} \end{cases}$$

A significant result (small p-value) indicates that the correlation structure is adequate for factor analysis.

Definition 11 (Kaiser-Meyer-Olkin (KMO) Measure)**.** The KMO statistic compares the magnitude of observed correlation coefficients to that of partial correlation coefficients. It is defined as:

$$\text{KMO} = \frac{\sum_{i \neq j} r_{ij}^2}{\sum_{i \neq j} r_{ij}^2 + \sum_{i \neq j} p_{ij}^2},$$

where r_{ij} are the pairwise correlations and p_{ij} the partial correlations. Values close to 1 indicate that factor analysis is likely to be useful, while values below 0.5 suggest it may not be appropriate.

2.4 Application to Portfolio Management: Alpha Research

In this section, we present an investment strategy leveraging factor analysis and alpha estimation. Using the factor loadings estimated from historical data, we develop a straightforward yet robust approach to identify potential mispricings or trading opportunities.

2.4.1 Introduction to Alpha

Recall our factor model equation, where asset returns can generally be decomposed into systematic and idiosyncratic components, expressed formally as:

$$\mathbf{x_t} = \boldsymbol{\alpha} + \boldsymbol{\Lambda}\mathbf{f_t} + \boldsymbol{\epsilon_t},$$

where $\mathbf{x}_t$ represents the asset returns, $\Lambda\mathbf{f_t}$ captures systematic risk exposures, and $\boldsymbol{\epsilon_t}$ denotes idiosyncratic risk. Initially proposed by Jensen (1968), the term α can be associated with abnormal returns not explained by common market factors, serving as a key indicator of potential asset miss-pricing.

Definition 12 (Alpha)**.** The alpha of an asset is defined as the expected return after removing the contributions of systematic risk factors:

$$\boldsymbol{\alpha} = \mathbf{x_t} - \Lambda\mathbf{f_t}.$$

Identifying meaningful alpha is crucial for asset selection, portfolio construction, and performance attribution.

Properties 3. Some important properties include, see Paleologo (2025):

i. Given an estimated alpha vector $\boldsymbol{\alpha} \in \mathbb{R}^n$ and factor loading matrix $\Lambda \in \mathbb{R}^{n \times m}$, the *spanned alpha*, α_{s}, is the projection of α onto the column space of Λ: $\alpha_{\mathrm{s}} = \Lambda(\Lambda'\Lambda)^{-1}\Lambda'\alpha$.

ii. $\alpha_{\mathrm{s}} \in \mathrm{Col}(\Lambda)$, meaning it is fully captured by systematic factors.

iii. Then the vector alpha can be decomposed into *spanned alpha* and *orthogonal alpha*, such that:

$$\alpha \;=\; \alpha_s \;+\; \alpha_\perp$$

For the purposes of this project, we focus specifically on the orthogonal component of alpha, representing unique asset-specific returns independent of systematic factors.

2.4.2 Alpha Estimation Method

We estimate alpha by regressing each asset's returns on factor scores obtained from historical data. Precisely, we estimate the following regression:

$$\mathbf{x}_{i,t} = \alpha_i + \Lambda_i f_t + \epsilon_{i,t}, \quad \text{for each asset } i,$$

where the intercept $\hat{\alpha}_i$ captures asset-specific abnormal returns. The significance of $\hat{\alpha}_i$ is tested at a 5% significance level to ensure robustness.

Definition 13 (Orthogonal Alpha)**.** The orthogonal alpha $\alpha_\perp$ is defined as the residual of alpha after projecting onto the space of factor loadings:

$$\alpha_\perp = \alpha - \Lambda(\Lambda'\Lambda)^{-1}\Lambda'\alpha.$$

2.4.3 Proposed Investment Strategy

Leveraging orthogonal alpha, we propose two investment strategies, on the one hand, a *long-only* investment portfolio comprised of the top 10 stocks in terms of estimated orthogonal alpha, and on the other hand, a *long-short* investment portfolio.

Proposition 2 (1)**.** [Long-Only Orthogonal Alpha-Based Strategy] Let $\hat{\alpha}_{\perp,i}$ denote the estimated orthogonal alpha for asset i, then the proposed *long-only* strategy consist of the following:

1. For each asset, estimate orthogonal alpha $\hat{\alpha}_{\perp,i}$ over the estimation window (January 2000 – December 2018, around 70% of available data).
2. Rank assets based on $\hat{\alpha}_{\perp,i}$, selecting those whose estimated alpha is statistically significant (p-value < 0.01).
3. Form a long-only portfolio consisting of long the top decile (top 10%) of ranked assets.
4. Assign equal weights within each selected portfolio segment.

Proposition 3 (2)**.** [Long-Short Orthogonal Alpha-Based Strategy] Using the estimated $\hat{\alpha}_{\perp,i}$, the *long-short* portfolio consists of:

1. Form a long/short portfolio consisting of long the top decile (top 10%) and short the bottom decile (bottom 10%) of ranked assets based on statistically significant (p-value < 0.01) estimated orthogonal alpha.

2. Assign equal weights within each selected portfolio segment.

We will backtest the strategy from January 2019 to December 2024 using the remaining 30% of the data, and will compare the results based on the following standard investment assessment metrics:

1. Annual mean yearly return $\overline{X_i}$ and annual standard deviation of returns σ_i
2. Risk-adjusted returns, also known as the Sharpe Ratio, see Sharpe (1964), and calculated as:
$$SR_i = \frac{\overline{X_i}}{\sigma_i},$$

Remark 6. This strategy does not account for transaction costs, market impact, or operational constraints, which represent a limitation of this backtesting methodology.

3 Numerical Application

3.1 Proposed Methodology

In order to conduct the proposed exploratory factor analysis of asset returns, we have develop the following methodology, which encompasses all of the previously described mathematical methods. This process unfolds through eight essential steps, each contributing critical insights into the data and validating the adequacy of the factor model.

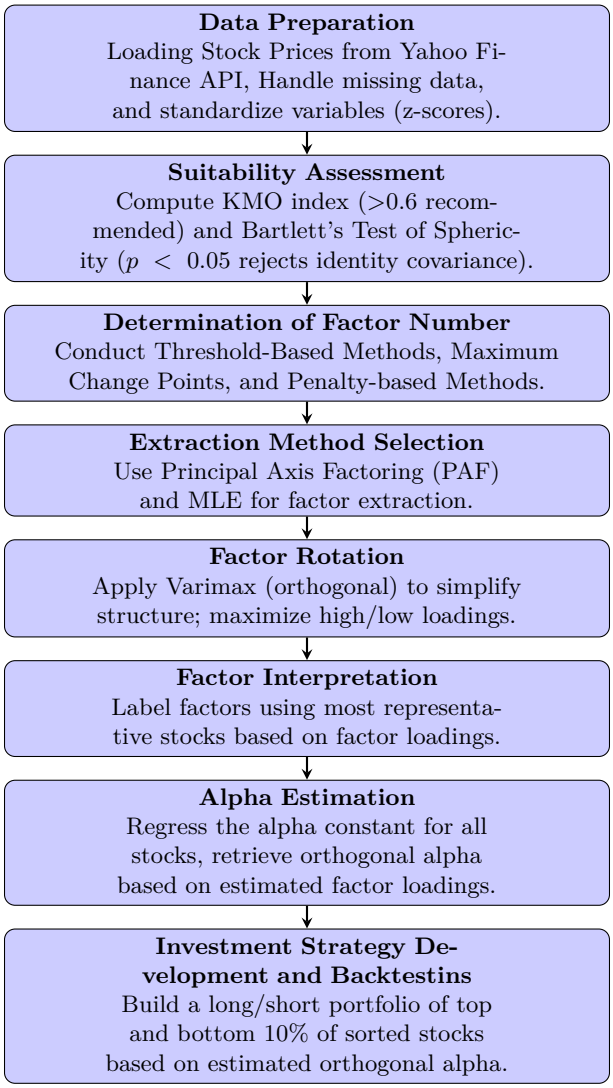


Figure 1: Enhanced EFA Methodology with Portfolio Management Application

3.2 Data Collection and Pre-processing

3.2.1 Collecting, Processing and Assessing Excess Log-Returns

Constituent tickers of the S&P 500 index were retrieved from the public Wikipedia listing. Daily adjusted closing prices were then downloaded from Yahoo Finance for the period spanning January 1, 2000 to December 31, 2018. Out of the full set, 151 stocks exhibited missing values and were excluded from the analysis. The final dataset contains 352 stocks with complete return histories.

Excess log-returns were computed by subtracting a proxy for the risk-free rate—the U.S. 10-Year Treasury yield (US10Y)—from the log-differenced price series.

Figure 2: U.S. Treasury 10 Year Bond Yields(%)

Interestingly, as Figure 2 shows, the U.S. 10-Year Treasury yield (US10Y) over the sample period reflects the macro-financial richness of the 2000–2018 window used for factor extraction. It spans several major economic cycles, including the dot-com bubble, the 2008 global financial crisis, and the early stages of post-crisis recovery. As such, it provides a dynamic backdrop against which latent factor structures are likely to evolve meaningfully.

Similarly, Figure 3 offers a concise overview of the descriptive statistics of individual asset returns. On the left-hand side, average daily returns cluster near zero with minimal dispersion, suggesting uniform performance across stocks. Minimum returns show a significant dispersion with an important downward skew, indicating the presence of sharp declines. Maximum returns show more modest gains with few extreme positive values. On the right-hand side, the correlation matrix reveals weak co-movement in daily returns, with most correlations ranging between 0.0002 and 0.0008. The diagonal captures each asset's variance, highlighting the overall independence in the dataset.

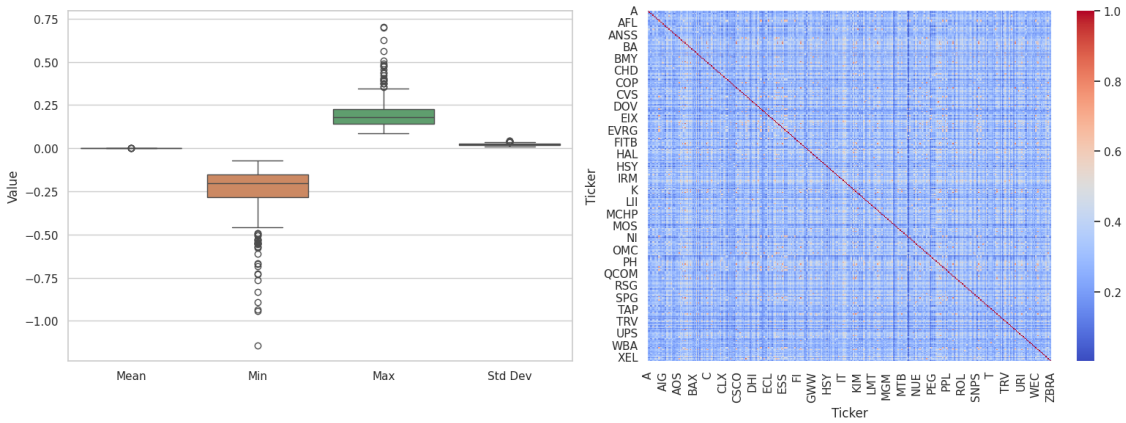


Figure 3: Descriptive stats of returns (lhs), Covariance Matrix (rhs)

3.2.2 Suitability Assessment

As explained in previous sections, we applied two standard statistical diagnostics to assess the adequacy of the data: the Kaiser-Meyer-Olkin (KMO) measure of sampling adequacy and Bartlett's Test of Sphericity. The following table summarizes the results.

Test	**Value**
Kaiser-Meyer-Olkin (KMO)	0.9936
Bartlett's Test Chi-square	1,202,106.69
Bartlett's Test p-value	< 0.000001

Table 2: *Suitability Metrics for Factor Analysis*

The global KMO value of 0.9936 indicates excellent sampling adequacy, supporting the presence of common factors. The extremely small p-value from Bartlett's test strongly rejects the null hypothesis of identity in the correlation matrix, further validating the use of factor models. The histogram in Figure 4 displays the distribution of individual KMO statistics across stocks. Most values cluster tightly above 0.99, reinforcing the conclusion that the dataset exhibits strong inter-variable correlations and is highly suitable for factor analysis.

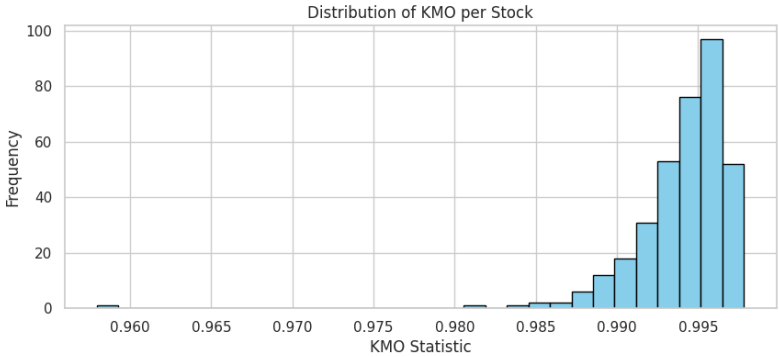


Figure 4: Distribution of the KMO Metric per Stock

3.2.3 Determining the Number of Latent Factors

We apply the three standard selection criteria described, namely: the Threshold Method, the Maximum Change Point Method, and the Penalty-Based Method. Each criterion reflects a distinct balance between complexity and model fit.

Selection Method	**Number of Factors**
Threshold Method	25
Maximum Change Point Method	114
Penalty-Based Method	342

Table 3: *Number of Factors Selected by Method*

As we can see in Table 3, every method suggests very different number of factors; among these, the Threshold Method proposes only 25 latent factors, which is convenient for the purpose our study. This result is particularly attractive from a modeling and computational standpoint, as it reduces dimensionality while retaining a significant proportion

of explanatory variance. In contrast, the Penalty-Based and Maximum Change Point methods suggest more complex structures that may lead to overfitting or redundancy in practical applications.

3.3 Model Estimation: Principal Axis Factoring Method

3.3.1 Systematic Explained Variance

Factor estimation was conducted using the Principal Axis Factoring (PAF) method via the `Factor` function from the Python library `statsmodels`, specifying the option `method='pa'`.

The explanatory power of the extracted 25 factors is detailed in Table 2. Similarly, Figure 5 presents the scree plot, illustrating the percentage of variance explained by each factor. A pronounced "elbow" at the first few factors underscores their relative importance, with diminishing incremental variance explained thereafter.

Factor	Explained Variance (%)	Cumulative Explained Variance (%)
1	32.15	32.15
2	3.81	35.95
3	2.73	38.69
4	1.93	40.61
5	1.79	42.41
6	1.21	43.62
7	1.08	44.70
8	0.76	45.46
9	0.64	46.10
10	0.51	46.61
11	0.47	47.07
12	0.42	47.49
13	0.40	47.89
14	0.37	48.26
15	0.33	48.59
16	0.31	48.91
17	0.30	49.21
18	0.29	49.50
19	0.28	49.78
20	0.26	50.04
21	0.25	50.29
22	0.23	50.52
23	0.23	50.75
24	0.22	50.97
25	0.20	51.17

Table 4: *Variance Explained by Extracted Factors*

It is noteworthy that the cumulative variance explained by all 25 factors is approximately 51%; this could appear low, based on our PCA-view of the factor model, but this moderate explanatory power is characteristic of factor models, as they intentionally isolate common systematic variation from asset-specific idiosyncratic noise, thus representing a deliberate modeling feature rather than a limitation.

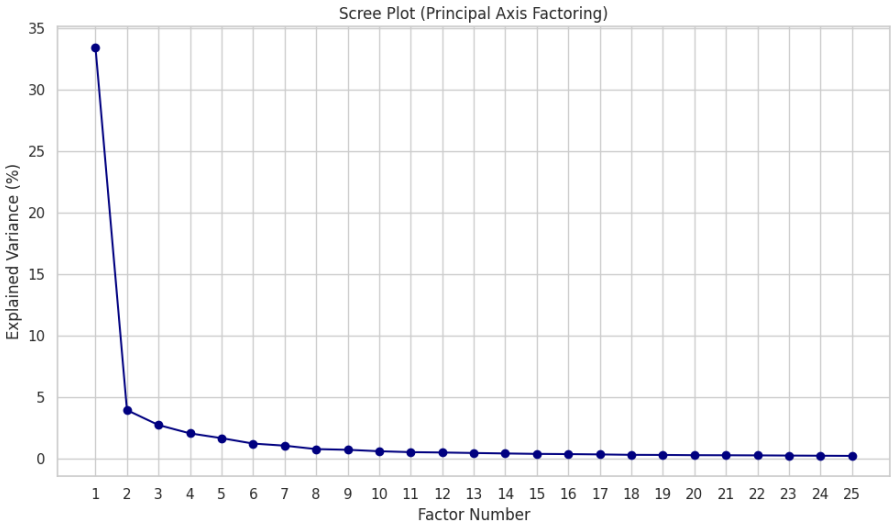


Figure 5: Selecting Number of Factor: Scree Plot

3.3.2 Assumptions & Estimation Performance

Another important output of our factor model estimation is the diagonal *uniqueness* covariance matrix $\mathbf{\Psi}$ shown in Figure 7. As expected the matrix is a diagonal matrix of ideosyncratic variances. Similarly, the assessment of the quality of our estimation computed through the *Mean Absolute Residual* indicates a good model fit; as we can see in Table 5, the residual es 0.0081.

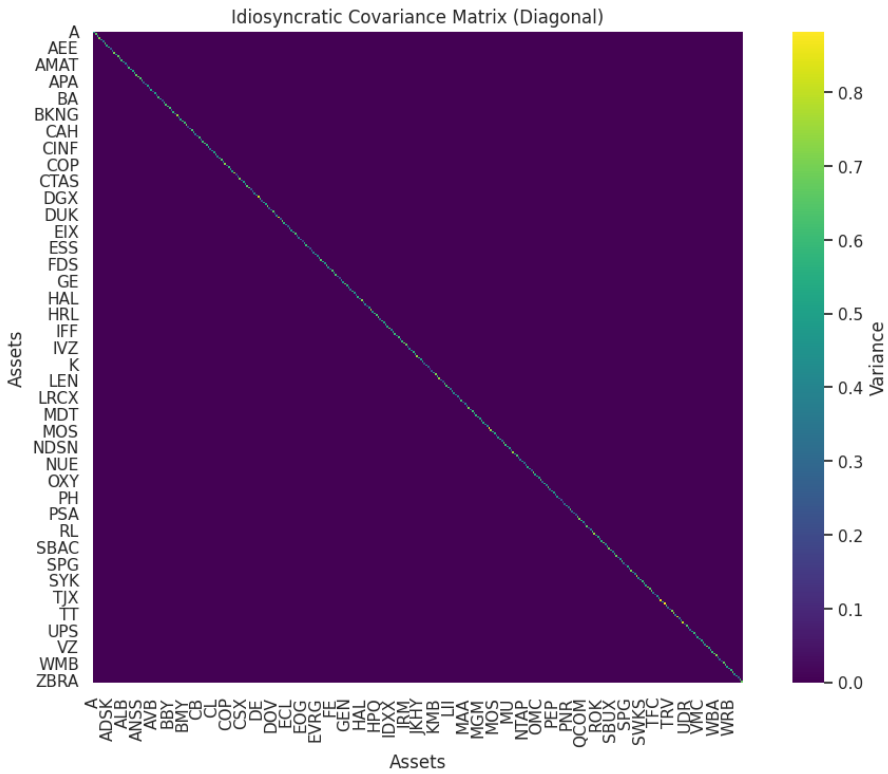


Figure 6: Estimated Uniqueness/Ideosyncratic Covariance Matrix

Metric	Value
Mean Absolute Residual Correlation	0.0081

Table 5: *Quality of Model Fit*

3.4 Factor Loadings Rotation

Figures 6 depicts heatmaps of factor loadings before and after applying Varimax rotation, respectively. Initially, factor loadings exhibit diffuse patterns across factors, indicating overlapping contributions, with a clear preponderance of the first factor, one which is often associated with the so-called "Market" (Paleologo, 2025). As we can see, the Varimax rotation substantially clarifies these patterns, yielding factors with distinctly concentrated positive and negative loadings, enhancing interpretability.

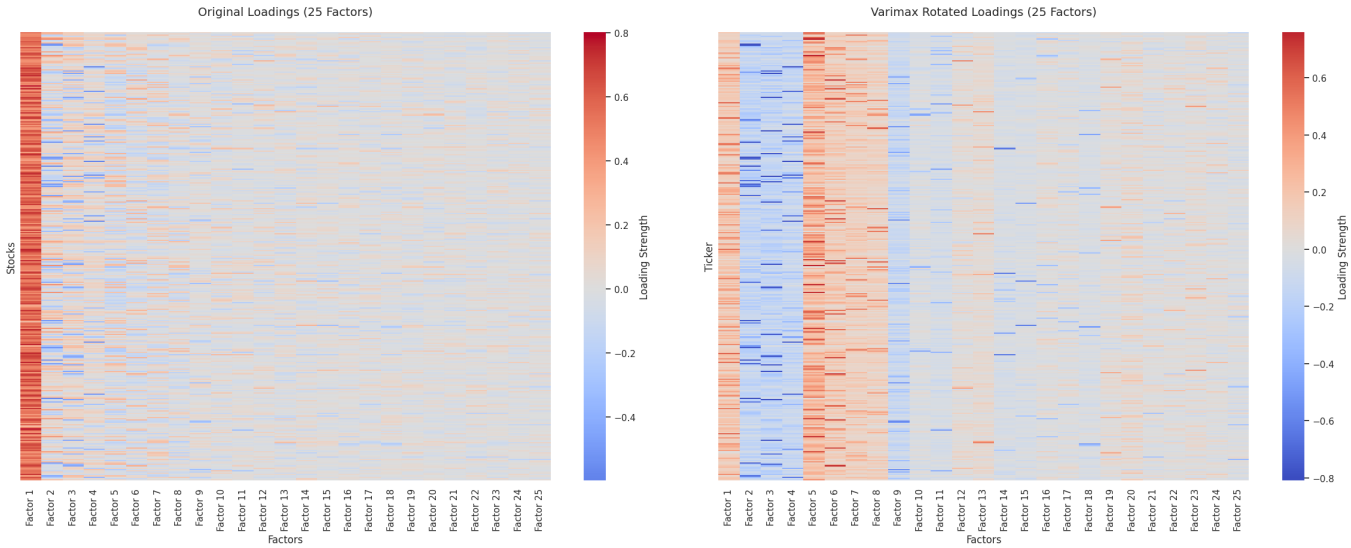


Figure 7: Initial Factor Loadings (lhs) vs. Varimax Loadings (rhs) for All Stocks

3.5 Latent Factor Interpretation

Interpreting explored latent factors can be quite challenging, but it is as important as the estimation itself, as it gives meaning and clarity to the factor model. We propose to interpret our estimated factor through the lens of the the most common sector of its most representative stocks.

Factor	Tickers	Sector
1	PH, CAT, TT, ITW, PPG	Industrial and Manufacturing
2	ED, SO, AEP, DTE, AEE	Utilities and Public Services
3	EQR, CPT, AVB, FRT, REG	Real Estate Investment Trusts (REITs)
4	APA, DVN, EOG, HES, COP	Energy and Oil
5	AMAT, KLAC, LRCX, ADI, TXN	Technology and Semiconductors
6	BAC, WFC, RF, KEY, FITB	Banking and Financial Services
7	BIIB, GILD, VRTX, INCY, REGN	Biotechnology and Pharmaceuticals
8	CL, CLX, PG, KMB, CHD	Consumer Goods and Staples
9	TGT, LOW, WMT, ROST, HD	Retail and Discount Stores
10	K, GIS, CPB, HRL, CAG	Packaged Food and Beverages
11	CB, EG, TRV, WRB, AON	Insurance and Financial Services
12	AMT, SBAC, CCI, PWR, WMB	Telecommunications and Utilities
13	HUM, UNH, CI, UHS, HIG	Healthcare Services
14	PHM, DHI, LEN, NVR, MAS	Homebuilding and Construction Materials
15	RCL, CCL, MGM, MAR, TPL	Leisure, Travel, and Hospitality
16	CHRW, UNP, NSC, CSX, EXPD	Transportation and Freight Logistics
17	NEM, FCX, MOS, STLD, NUE	Basic Materials and Industrial Metals
18	MRK, PFE, LLY, ABT, JNJ	Pharmaceutical and Healthcare Supply
19	LMT, NOC, GD, BA, TDY	Aerospace and Defense
20	LH, DGX, UHS, DVA, COR	Healthcare Distribution and Diagnostics
21	TYL, COO, WST, CSGP, FICO	Niche Software, Analytics, Healthcare Tech
22	VZ, T, PCG, EIX, DECK	Telecom and Utility Infrastructure
23	CAH, COR, MCK, CVS, HOLX	Healthcare Retail and Wholesale
24	WAT, RVTY, TMO, MTD, A	Scientific and Medical Technology
25	MU, TER, LRCX, INTU, KLAC	Semiconductors and High-Tech

Table 6: *Sector Classification of Most Representative Stocks per Factor*

3.6 Alpha Estimation

To estimate alpha, we proceeded to compute an Ordinary Least Squares (OLS) regression on the matrix of asset returns $\mathbf{X}$ and the estimated factor loadings $\widehat{\mathbf{\Lambda}}$ and factor returns $\widehat{\mathbf{F}}$. We used the `OLS` function of the `statsmodel` library, and explicitly admitted the model with a constant term (i.e. the alpha).

The two tables present the top and bottom ten stocks as ranked by their estimated orthogonal alpha, based on our factor model. These results reflect the output of the model after projecting raw alpha estimates onto the orthogonal complement of the factor space, thereby isolating asset-specific returns not explained by systematic risk.

The top-ranked assets (Table 7) include companies such as United Rentals (URI), Textron (TXT), and Cummins (CMI). Their high orthogonal alpha suggests that their recent performance cannot be fully attributed to common market factors, potentially reflecting idiosyncratic advantages or market underpricing.

Conversely, the bottom-ranked assets (Table 8) highlight stocks such as NVIDIA (NVDA), Illinois Tool Works (ITW), and 3M (MMM), all of which show significantly negative orthogonal alpha. Of particular note is NVIDIA, a high-profile stock widely recognized for its role in the semiconductor and AI sectors. Its negative alpha estimate could reflect recent overvaluation relative to fundamentals, saturation of investor sentiment, or the model's inability to fully capture sector-specific dynamics driving its price, which may still be linked to broader thematic trends rather than stock-specific fundamentals.

Ticker	Alpha	Comment
URI	0.1361	United Rentals – Equipment leasing
TXT	0.1207	Textron – Aerospace and defense
CMI	0.0932	Cummins – Engines and power systems
TDY	0.0818	Teledyne – Industrial electronics
STLD	0.0758	Steel Dynamics – Steel production
DD	0.0753	DuPont – Specialty chemicals
FCX	0.0714	Freeport-McMoRan – Copper and gold mining
ODFL	0.0694	Old Dominion – Freight logistics
TRMB	0.0667	Trimble – Geospatial technology
MOS	0.0646	Mosaic – Agricultural chemicals

Table 7: *Top 10 Stocks by Estimated Alpha with Company Description*

Ticker	Alpha	Comment
VRTX	-0.0609	Vertex – Biotechnology
ECL	-0.0649	Ecolab – Water and hygiene solutions
VRSN	-0.0654	Verisign – Internet infrastructure
WDC	-0.0710	Western Digital – Data storage
HUBB	-0.0728	Hubbell – Electrical products
UPS	-0.0834	UPS – Parcel logistics
MMM	-0.0875	3M – Industrial conglomerate
ITW	-0.0893	Illinois Tool Works – Manufacturing
GPC	-0.1012	Genuine Parts – Automotive and industrial parts
NVDA	-0.1073	NVIDIA – Semiconductors and AI hardware

Table 8: *Bottom 10 Stocks by Estimated Alpha with Company Description*

These results underscore the strength of orthogonal alpha in identifying return components not attributable to known systematic factors. In the following section we present the results from the proposed long/short portfolio of these stocks.

3.7 Alpha Strategy Backtest

i. Long-Short Portfolio

The table below reports the key performance metrics of the proposed strategy, both in-sample (2000–2018) and out-of-sample (2019–2024), benchmarked against the S&P 500 index. As mentioned earlier, metrics include annualized mean return, volatility, and Sharpe Ratio, enabling a comparative assessment of risk-adjusted performance over both estimation and testing periods.

As we can see the *long-short* portfolio significantly under-performs the market across all metrics and periods. In-sample, the strategy delivers an annual return of 1.90% with a Sharpe Ratio of 0.10, compared to 4.78% and 0.25 for the S&P 500. Out-of-sample performance shows the similar behavior, with the portfolio achieving 2.13% return and a Sharpe Ratio of 0.13, versus 15.09% and 0.75 for the benchmark. These results suggest that the orthogonal alpha signal might not be appropriate for the construction of *long-short* portfolios expected to perform better than the benchmark across long-to-medium time frames. Although these results are not satisfactory, the stability of the performance of the portfolio across in and out-of-sample results, indicated that the model's estimated

parameters contain valuable information about unobservable latent factors of the studied stocks returns.

Metric	Portfolio	Market (S&P 500)	Period
Mean Annual Return	1.90%	4.78%	In-sample
Annual Volatility	19.57%	19.30%	In-sample
Sharpe Ratio	0.10	0.25	In-sample
Mean Annual Return	2.13%	15.09%	Out-of-sample
Annual Volatility	16.56%	20.17%	Out-of-sample
Sharpe Ratio	0.13	0.75	Out-of-sample

Table 9: *In-Sample and Out-of-Sample Performance Long-Short Portfolio vs Benchmark*

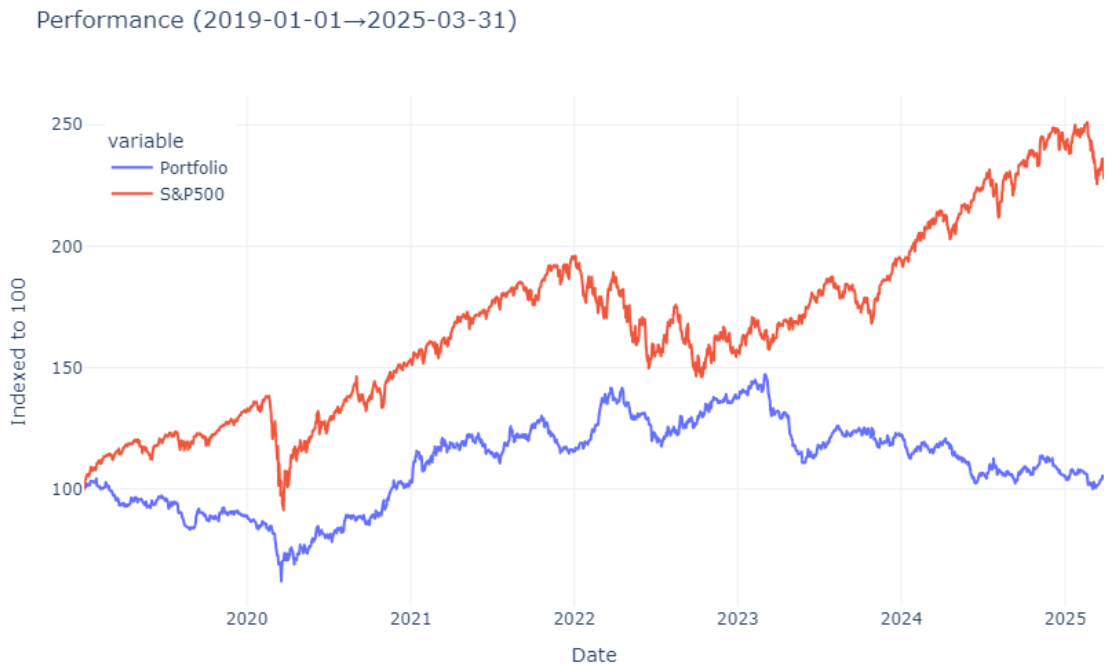


Figure 8: Long-Short Portfolio vs S&P500, 2019-2025 (Indexed to 100)

ii. Long-Only Portfolio

The table below reports the performance metrics of this portfolio in the same in-sample (2000–2018) and out-of-sample (2019–2024) databases. Again, the results are benchmarked against the S&P 500 index.

Contrary to the *long-short* portfolio, this strategy yields more interesting results across all metrics and periods. In-sample, the strategy delivers an annual return of 20.57% with a Sharpe Ratio of 0.69, compared to 4.78% and 0.25 for the S&P 500. Interestingly, out-of-sample performance shows again very stable results; the portfolio achieves a 23.02% return and a Sharpe Ratio of 0.78, versus 15.09% and 0.75 for the benchmark.
Although these results are promising it is important to highlight that the back-test does not take into account the possible cost of holding single stocks versus and marked-index

Exchange Traded Fund. Nevertheless, the proposed *long-only* investment strategy based on the statistical model's estimated orthogonal alpha merits further studies. We will discuss this in the conclusion section of the project.

Metric	**Portfolio**	**Market (S&P 500)**	**Period**
Mean Annual Return	20.57%	4.78%	In-sample
Annual Volatility	29.83%	19.30%	In-sample
Sharpe Ratio	0.69	0.25	In-sample
Mean Annual Return	23.02%	15.09%	Out-of-sample
Annual Volatility	29.40%	20.17%	Out-of-sample
Sharpe Ratio	0.78	0.75	Out-of-sample

Table 10: *In-Sample and Out-of-Sample Performance Long-Only Portfolio vs Benchmark*

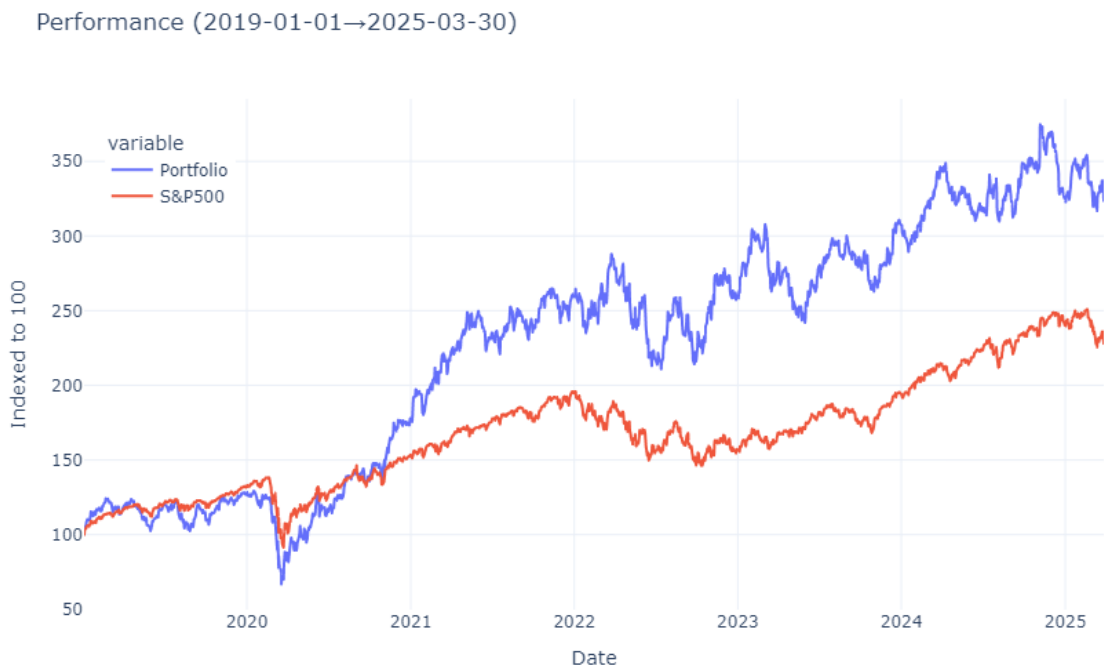


Figure 9: Long-Only Portfolio vs S&P500, 2019-2025 (Indexed to 100)

4 Discussion & Conclusion

The project showcased how statistical factor analysis represents a useful methodology in modern quantitative finance, offering researchers and practitioners a powerful framework for exploring complex datasets, where the presence of unobservable latent factors drives observable variables—in our case, stock returns. This short yet concise study illustrated how EFA, a well-established statistical tool, can help us find, interpret, and apply these latent structures.

As presented earlier, we estimated 25 latent factors of price returns for 352 constituent stocks of the S&P500. Nevertheless, the choice of 25 factors was not straightforward, as different factor-selection criteria yielded significantly different results. In Table 3 we saw that the three methods—threshold, maximum change-point, and penalty-based—identified 25, 114, and 342 as the optimal number of factors, respectively. Ideally, these methods would converge on similar numbers; their divergence underscores how heuristic approaches can be hard to implement in practice and will eventually require some discretionary judgement from practitioners and researchers. In our case, the choice of 25 factors was based on convenience and practicality, but further assessment could lead to better decision-making. In future studies, we could employ statistical tests, such as the likelihood-ratio test, which is well suited given that simpler models (i.e., fewer factors) are nested inside more complex ones.

Another notable finding of the project was the suitability of the factor model for the asset-return data; both the sampling adequacy (KMO value = 0.9936) and the rejection of sphericity (Bartlett's test p-value < 0.000001) confirmed the presence of significant factor structure within the dataset. This motivates us to believe that further analysis and refinement of statistical factor models of returns are indeed worthwhile, both for research and for practical application.

Regarding the estimation methods, during this project we were only able to implement PAF, owing to computational and time constraints. We attempted the maximum-likelihood estimation of the model with 25 factors, but after more than four hours of computation we decided to focus on and improve the PAF estimation. Although PAF is sometimes considered less robust (see Paleologo (2025)), it remains useful because it does not require any a-priori assumption about the probability distribution of the return matrix. The `statsmodels` package and the `Factor` library proved to be highly useful and precise; this was confirmed by examining the *uniqueness* covariance matrix $\mathbf{\Psi}$—which was diagonal as expected—and by the minimal *Mean Residual Error*.

Another interesting outcome was the application of the *rotational indeterminacy* theorem through a *Varimax* rotation. As explained earlier, this kind of orthogonal rotation preserves the factor structure while improving interpretability by pushing loadings either very high or near zero. As shown in Figure 6, the factor structure became far more interpretable after rotation. Further studies are necessary, especially to test and compare the Varimax rotation with other orthogonal criteria (e.g., *Quartimax*) and with oblique rotations such as *Promax*. The project also deepened our understanding of how EFA differs from other "descriptive" dimensionality-reduction techniques, such as *Principal Component Analysis*. Whereas PCA typically seeks very high explained variance, in factor analysis a modest 51 % explained variance was satisfactory because of the importance

of variance partitioning between $\mathbf{\Lambda\Lambda'}$ and $\mathbf{\Psi}$.

Although applying statistical methods for their own sake is intellectually stimulating, their economic interpretation and application remained our main concern. As shown earlier, we identified the most representative economic sector of stocks for each of the 25 extracted factors, yielding valuable insights into the underlying structure of market returns. Each factor demonstrated an association with specific industry groupings. Across the 25 factors we identified clear sector themes—industrials, utilities, real estate, energy, technology, banking, biotech, and other groups such as consumer staples, transportation, defense, and semiconductors. Clustering factor association with qualitative data can certainly be improved, but this initial assessment already illustrates how sector-based commonalities can drive systematic return patterns.

Perhaps the most important results stem from applying the statistical factor model to portfolio management—specifically, to alpha research. As shown previously, of the two proposed investment strategies (the *long–short* and the *long-only* portfolios), only the latter displayed promising results based on basic performance metrics; nevertheless, the outcomes of both portfolios provide valuable lessons and motivation for further study. When assessing alpha strategies, one must always ask whether the proposed approach is truly harvesting alpha or merely repackaging some form of market "beta", see Ardia (2024). Table 11 reports the weighted CAPM betas of the two portfolios. For the long–short portfolio, the beta is 0.20. Using the standard approximation $\beta = \rho_{A,M}\frac{\sigma_A}{\sigma_M}$, such a low beta implies expected annual returns below 0.9 %, which is less than both the in- and out-of-sample returns. This raises the question of whether the estimated alpha is genuinely delivering abnormal excess returns.

Metric	**Long–Short**	**Long-Only**
CAPM Beta	0.20	1.20

Table 11: *CAPM beta comparison.*

This question—whether the statistical factor model can truly help us uncover unique investment opportunities that generate abnormal returns—is at least partially answered by our results. As noted, the *long–short* portfolio exhibits a significantly low market beta yet still achieves above-expected returns and a healthy Sharpe ratio, encouraging us to explore the construction of a market-neutral strategy based on factor-analysis insights. The conviction is reinforced by the *long-only* strategy: as Table 12 indicates, the portfolio comprises a diverse set of stocks with low-to-high betas which, in an equal-weighted portfolio, yield a market beta of approximately 1.20. This moderate exposure is promising because it suggests that the strategy's outperformance is not simply the consequence of amplified market risk; rather, the returns appear to arise from identifying stocks with positive idiosyncratic components that common factors do not explain. As for the *long-short* strategy, this observation should prompt us to question whether the market index is the most appropriate benchmark for the long–short strategy and whether alternative benchmarks would be more suitable, to actually compare the proposed portfolio. We leave this question open for future investigation.

Ticker	Beta
URI	1.719
TXT	1.064
CMI	1.069
TDY	1.028
STLD	1.359
DD	1.243
FCX	1.708
ODFL	1.256
TRMB	1.576
MOS	1.053

Table 12: *Betas of stocks comprising the long-only portfolio.*

To further evaluate the viability and robustness of the proposed strategies, a rolling back-testing protocol should be implemented. Such an approach would re-estimate the factor model at regular intervals (e.g., monthly or quarterly) using rolling windows of historical data, then form portfolios based on the latest alpha estimates. This methodology would better accommodate the time-varying nature of factor structures and alpha opportunities, mitigating look-ahead bias, data leakage, and parameter instability. A realistic rolling back-test would also incorporate transaction costs, market-impact considerations, and other implementation constraints.

On a final note, our study of asset returns over the 2000–2025 period is particularly compelling because it spans the whole of recent economic history, including several distinct market regimes and events, such the dot-com collapse, the 2008 global financial crisis, the quantitative easing period, the COVID-19 pandemic, and the current escalation of U.S. Trade Wars. This encourages us to recognize the stability of our results in and out-of-sample, suggesting, at the very least, that factor analysis can capture robust signals useful for quantitative portfolio management. Yet, further work is required to develop a fully implementable strategy, including rolling factor loadings, more robust alpha orthogonalisation, and, of course, the inclusion of realistic transaction costs.

References

Ardia, D. (2024). Is it alpha or beta? decomposing hedge fund returns when models are misspecified.

Bai, J. and Li, K. (2012). Statistical analysis of factor models of high dimension. *The Annals of Statistics*, 40(1).

Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236.

Fama, E. F. and French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56.

Harman, H. H. (1967). *Modern factor analysis*. University of Chicago Press. Accessed: 13 April 2025, via National Library of Australia.

Jensen, M. C. (1968). The performance of mutual funds in the period 1945–1964. *The Journal of Finance*, 23(2):389–416.

Ledoit, O. and Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411.

Lintner, J. (1965). The valuation of risk assets and the selection of risky investments in stock portfolios and capital budgets. *Review of Economics and Statistics*, 47(1):13–37.

López de Prado, M. (2022). Causal factor investing: Can factor investing become scientific? *SSRN Electronic Journal*.

Onatski, A. (2005). Determining the number of factors from empirical distribution of eigenvalues. *Review of Economics and Statistics*, 92.

Paleologo, G. A. (2025). *The elements of quantitative investing*. John Wiley & Sons, Nashville, TN.

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572.

Ross, S. A. (1976). The arbitrage theory of capital asset pricing. *Journal of Economic Theory*, 13(3):341–360.

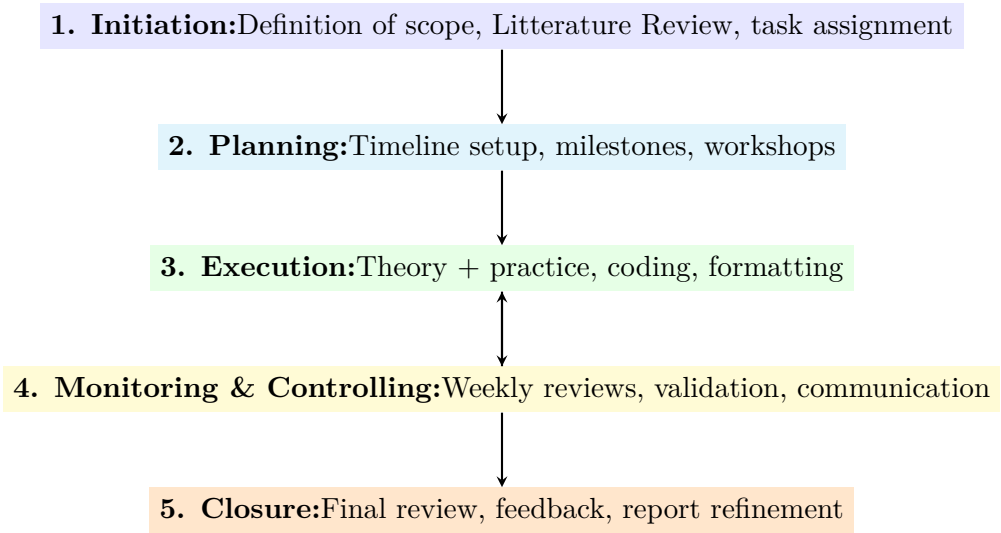
Sharpe, W. F. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance*, 19(3):425–442.

Spearman, C. (1904). General intelligence, objectively determined and measured. *American Journal of Psychology*, 15(2):201–292.

Thurstone, L. L. (1947). *Multiple Factor Analysis: A Development and Expansion of the Vectors of Mind*. University of Chicago Press.

A Appendix A – Project Management & Organization

To successfully carry out our project on *An Initial Evaluation of Exploratory Factor Analysis for Alpha Research*, we decided to plan and structure our approach to ensure project coherence, meet deadlines, optimize collaboration among group members, and leverage everyone's strengths to achieve a strong final result. The organization of the project was structured as follows:



1. Initiation

Objectives and Scope

We set two goals: (1) review factor-model theory (PCA, PAF, MLE, Factor Interpretation, Alpha Research) and (2) apply EFA to daily excess log-returns of 352 S&P 500 stocks to extract latent factors and develop alpha strategies. A very simple project management framework was applied.

2. Planning

Schedule

- **Weeks 1–2**: Literature review; draft theoretical framework; workshop to align methods. During the first weeks a lot of work was done to actually understand the model we were working on, as well as the data we were working on. We held several workshops at ISFA's labs to redact the mathematical framework by hand; this was crucial for us as it we were convinced that understanding the foundations of the model was more important than the computational results.
- **Week 2**: Assessment with the team's Academic Supervisor. During this meeting, the team did not have a clear methodology, nor objectives, the meeting was fruitful and allowed us to move forward.
- **Weeks 3–5**: Download prices; compute excess log-returns; run KMO and Bartlett tests.
- **Weeks 6–8**: Implement PAF and MLE; buffer time for memory issues. After the first model estimation we shared the results with our supervisor.

- **Weeks 9–10**: Interpret factors; apply Varimax rotation; estimate alpha signals.
- **Weeks 11–12**: Construct and backtest long-only and long–short portfolios; group review in Colab. During these weeks, a lot of back-and-forth was necessary because the initial code was giving erroneous results. Luckily we were able to correct the code and assess these results.

Team Roles

- **Manuel - Project Lead**: Document redaction, citations, abstract/conclusion; coded the Varimax rotation, alpha estimation, alpha orthogonalization, and portfolio backtesting.
- **Manuel, Abdoul Barik & Anthony**: Factor estimation in `statsmodels` and empirical analysis. We all worked on this section of the project, specifically to assess and audit each others code.
- **Abdoul Barik**: Factor interpretation through economic sector clustering.
- **All**: Shared Google Colab notebooks; regular code reviews for reproducibility.

3. Execution

Monitoring

Weekly meetings compared progress against the planned schedule, confirmed factor-selection methods, and resolved technical issues. Each week, we held from 2-3 short meetings between classes, to assess progress in both the code and the documents.

Communication

A WhatsApp group handled quick fixes and task reassignments; LaTeX drafts circulated by email.

4. Control

Risk and Quality Assurance

We anticipated potential bottlenecks—such as computational delays during MLE—and integrated buffer periods into the schedule. Quality checks included cross-validation of factor-selection methods and inspection of uniqueness matrices to confirm diagonal structures. Any deviations triggered immediate review sessions to mitigate risks.

5. Supervisor Feedback

Manuel served as liaison with our supervisor, Mrs. Ying, organizing bi-weekly meetings and email updates. Her guidance prompted us to justify the selection of 25 factors over larger alternatives and to contextualize our Sharpe-ratio findings economically. This iterative feedback refined our analysis and ensured alignment with both theoretical expectations and practical investment considerations.

B Appendix B – Python source code

B.1 Data Cleaning and Processing

```python
"""alphaefa.io      data acquisition & basic  p r e processing
=====================================================
This module now **consolidates** all  a d hoc  helper functions you had in the
notebook while keeping a clean public API.  Everything here concerns
*getting* data (prices,  r i s k free ), *inspecting* it (NA report), and f i r s t stage
*transformations* (log returns, winsorisation, standardisation). Analytics,
portfolio or plotting logic will live in other  s u b modules  (factor.py,
backtest.py, plots.py    ) to keep responsibilities clear.
"""
from __future__ import annotations

from datetime import datetime
from typing import Iterable, List

import numpy as np
import pandas as pd
import pandas_datareader.data as web
import yfinance as yf
from sklearn.preprocessing import StandardScaler

# Optional heavy imports guarded     only executed when functions are used
try:
    import seaborn as sns  # type: ignore
    import matplotlib.pyplot as plt  # type: ignore
except ModuleNotFoundError:  # pragma: no cover     OK for headless unit tests
    sns = plt = None  # type: ignore

__all__ = [
    #           fetching
    "get_sp500_tickers",
    "get_stock_prices",
    "get_riskfree_daily",
    #           inspection
    "na_description",
    #           transformations
    "compute_log_returns",
    "compute_excess_log_returns",
    "winsorize_matrix",
    "standardize_returns",
    #           quick descriptive plot
    "describe_and_plot",
]

```

```
#
    ---------------------------------------------------------------------------
# Fetching helpers
#
    ---------------------------------------------------------------------------

def get_sp500_tickers(*, user_agent: str | None = "alphaefa-bot") -> List[
    str]:
    """Scrape the current S& P 500  member list from Wikipedia."""
    import requests, pandas as _pd

    url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
    hdrs = {"User-Agent": user_agent} if user_agent else {}
    tables = _pd.read_html(requests.get(url, headers=hdrs).text)
    return tables[0]["Symbol"].str.upper().tolist()


def get_stock_prices(
    tickers: pd.DataFrame | Iterable[str],
    start_date: str | datetime,
    end_date: str | datetime,
    *,
    auto_adjust: bool = True,
    progress: bool = False,
) -> pd.DataFrame:
    """Download **Adj Close** price panel from Yahoo Finance.

    Parameters
    ----------
    tickers : DataFrame or iterable
        Either the Wikipedia table (as in your notebook) or a list/tuple
            of
        symbols.
    start_date, end_date : str | datetime
    auto_adjust : bool, default True
        Let yfinance adjust for splits / dividends.

    Returns
    -------
    DataFrame : index = dates, columns = tickers (uppercase)
    """
    if isinstance(tickers, pd.DataFrame):
        tickers = tickers["Symbol"].tolist()
    tickers = [t.upper().replace(".", "-") for t in tickers]

    px = yf.download(
        tickers,
        start=start_date,
        end=end_date,
        progress=progress,
        auto_adjust=auto_adjust,
        group_by="column",
    )
    if px.empty:
        raise ValueError("No␣data␣downloaded␣   ␣check␣tickers␣or␣dates.")

    # keep Adj Close if multiindex
    if isinstance(px.columns, pd.MultiIndex):
```

```python
        px = px["Adj␣Close"] if "Adj␣Close" in px else px[px.columns.levels[0][0]]

    px.columns = [c.upper().replace("-", ".") for c in px.columns]
    return px.sort_index()


# ---------------------------------------------------------------------------
# Risk free rate (FRED)
# ---------------------------------------------------------------------------

def _annual_to_daily(rate: float, trading_days: int = 252) -> float:
    return (1 + rate) ** (1 / trading_days) - 1


def get_riskfree_daily(
    start: str | datetime,
    end: str | datetime,
    *,
    fred_series: str = "DGS10",
    trading_days: int = 252,
) -> pd.Series:
    """Return a **daily** risk free series from FRED (default 10 year Treasury)."""
    rf = web.DataReader(fred_series, "fred", start, end).dropna()[fred_series] / 100
    return rf.apply(_annual_to_daily, trading_days=trading_days).rename("rf_daily")


# ---------------------------------------------------------------------------
# Inspection & NA report
# ---------------------------------------------------------------------------

def na_description(df: pd.DataFrame) -> pd.DataFrame:
    """Quick NA count and percentage for each column. No printing by default."""
    na_cnt = df.isna().sum()
    out = pd.DataFrame({
        "na_count": na_cnt,
        "na_pct": na_cnt / len(df) * 100,
    }).sort_values("na_pct", ascending=False)
    return out


# ---------------------------------------------------------------------------
# Transformations
# ---------------------------------------------------------------------------

```

```python
def compute_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
    """Daily  l o g returns  (1st diff of  l o g prices )."""
    return np.log(prices).diff().iloc[1:]


def compute_excess_log_returns(
    prices: pd.DataFrame,
    *,
    risk_free_rate: float | pd.Series = 0.0,
) -> pd.DataFrame:
    """ L o g returns  minus  r i s k free  (scalar or Series aligned on index)
        ."""
    lr = compute_log_returns(prices)
    return lr.sub(risk_free_rate, axis=0)


def winsorize_matrix(
    returns: pd.DataFrame,
    *,
    lower_quantile: float = 0.05,
    upper_quantile: float = 0.95,
    verbose: bool = True,
) -> pd.DataFrame:
    """ C o l u m n wise  winsorisation and optional summary printout."""
    wins = returns.copy()
    summary = {}
    for col in wins.columns:
        lo = returns[col].quantile(lower_quantile)
        hi = returns[col].quantile(upper_quantile)
        mask_lo = wins[col] < lo
        mask_hi = wins[col] > hi
        wins.loc[mask_lo, col] = lo
        wins.loc[mask_hi, col] = hi
        summary[col] = mask_lo.sum() + mask_hi.sum()

    if verbose:
        print("Winsorisation summary (counts):")
        for c, k in summary.items():
            print(f"{c}: {k} ({k / len(returns):.2%})")
    return wins


def standardize_returns(returns: pd.DataFrame) -> pd.DataFrame:
    """ Z score  each column (mean 0 , s t d 1)."""
    scaler = StandardScaler()
    std = pd.DataFrame(
        scaler.fit_transform(returns),
        index=returns.index,
        columns=returns.columns,
    )
    return std


# ---------------------------------------------------------------------------
# Quick descriptive stats & plot (exploratory only)
# ---------------------------------------------------------------------------

```

```python
def describe_and_plot(returns: pd.DataFrame):  # pragma: no cover plotting helper
    if sns is None or plt is None:
        raise ImportError("seaborn / matplotlib required for describe_and_plot().")

    stats = pd.DataFrame({
        "Mean": returns.mean(),
        "Min": returns.min(),
        "Max": returns.max(),
        "Std Dev": returns.std(),
    })

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    sns.boxplot(data=stats, ax=axes[0])
    axes[0].set_title(" C r o s s sectional distribution of basic stats")

    sns.heatmap(returns.corr(), cmap="coolwarm", ax=axes[1])
    axes[1].set_title("Correlation matrix")
    plt.tight_layout()
    plt.show()
    return stats
```

B.2 Factor Estimation

```python
"""alphaefa.factor     statistical  f a c t o r model  utilities
=========================================================
This  s u b module  takes standardised / winsorised *return* matrices produced by
`alphaefa.io` and provides:

* **Suitability diagnostics**     KMO & Bartlett
* ** D a t a driven  f a c t o r number  choices**     threshold /  m a x gap  / penalty
* ** P r i n c i p a l Axis  Factor (PAF) estimation**     thin wrapper over
  `statsmodels.multivariate.factor.Factor`
* **Rotation helpers**     Varimax (via *factor_analyzer*)
* ** E x p l a i n e d variance , residual diagnostics**

Plotting helpers are optional; they are imported lazily and guarded so the core
functions remain usable in headless environments.
"""
from __future__ import annotations

from typing import Dict, Literal, Tuple

import numpy as np
import pandas as pd
from statsmodels.multivariate.factor import Factor
from factor_analyzer.rotator import Rotator
from factor_analyzer.factor_analyzer import calculate_kmo, calculate_bartlett_sphericity

# Optional heavy plotting imports
try:
    import seaborn as sns  # type: ignore
    import matplotlib.pyplot as plt  # type: ignore
except ModuleNotFoundError:  # pragma: no cover
```

```
    sns = plt = None  # type: ignore

__all__ = [
    # diagnostics
    "suitability_assessment",
    # factor number rules
    "select_factors_threshold",
    "select_factors_max_change",
    "select_factors_penalty",
    "suggest_factor_numbers",
    # model fitting
    "fit_paf",
    "rotate_loadings",
    "explained_variance",
    # residuals
    "residual_correlation",
]

# ---------------------------------------------------------------------------
# Suitability diagnostics
# ---------------------------------------------------------------------------

def suitability_assessment(returns: pd.DataFrame, /) -> Dict[str, float]:
    """Compute **KMO** and **Bartlett** tests; optionally plot KMO histogram.

    Returns a dict with keys: ``kmo_model``, ``bartlett_chi2``, ``bartlett_p``.
    """
    kmo_all, kmo_model = calculate_kmo(returns)
    chi2, p_value = calculate_bartlett_sphericity(returns)

    out = {
        "kmo_model": float(kmo_model),
        "bartlett_chi2": float(chi2),
        "bartlett_p": float(p_value),
    }

    # Optional quick visual  only if seaborn available and in interactive mode
    if sns is not None and plt is not None and plt.isinteractive():
        plt.figure(figsize=(8, 4))
        sns.histplot(kmo_all, bins=30, color="skyblue", edgecolor="black")
        plt.title("Distribution of KMO per variable")
        plt.xlabel("KMO statistic")
        plt.tight_layout()
        plt.show()
    return out

# ---------------------------------------------------------------------------
# Number of factor selection rules
# ---------------------------------------------------------------------------

```

```python
def select_factors_threshold(eigenvalues: np.ndarray, gamma: float) -> int:
    """Rule: count eigenvalues     1 +      ."""
    threshold = 1 + np.sqrt(gamma)
    return int((eigenvalues >= threshold).sum())


def select_factors_max_change(
    eigenvalues: np.ndarray,
    *,
    use_log: bool = False,
    kmax: int | None = None,
) -> int:
    """Rule: index where successive eigenvalue *drop* is largest."""
    eig = eigenvalues[: kmax or len(eigenvalues)]
    diffs = np.diff(np.log(eig)) if use_log else np.diff(eig)
    return int(np.argmax(diffs) + 1)


def select_factors_penalty(eigenvalues: np.ndarray, n: int, T: int) -> int:
    """Bai & Ng (2002)  style   penalty criterion."""
    penalty = (n + T) / (n * T) * np.log((n * T) / (n + T))
    costs = [np.sum(eigenvalues[k:] ** 2) + k * penalty for k in range(len(eigenvalues) + 1)]
    return int(np.argmin(costs))


def suggest_factor_numbers(returns: pd.DataFrame) -> Dict[str, int]:
    """Convenience wrapper     compute eigenvalues & all three rules."""
    cov = returns.cov()
    eigvals = np.sort(np.linalg.eigvalsh(cov))[::-1]
    n, T = returns.shape[1], returns.shape[0]
    gamma = n / T
    return {
        "threshold": select_factors_threshold(eigvals, gamma),
        "max_change": select_factors_max_change(eigvals),
        "penalty": select_factors_penalty(eigvals, n, T),
    }

# ---------------------------------------------------------------------------
# PAF fitting & rotation
# ---------------------------------------------------------------------------


def fit_paf(
    returns: pd.DataFrame,
    n_factors: int,
    *,
    method: Literal["pa", "ml"] = "pa",
) -> Factor:
    """Wrapper around *statsmodels* ``Factor`` fit     returns fitted object."""
    factor = Factor(returns, n_factor=n_factors, method=method).fit()
    return factor


def rotate_loadings(
```

```python
    loadings: np.ndarray,
    *,
    method: str = "varimax",
    normalize: bool = True,
) -> np.ndarray:
    """Rotate a loadings matrix with *factor_analyzer* rotator."""
    rot = Rotator(method=method, normalize=normalize)
    return rot.fit_transform(loadings)

# ---------------------------------------------------------------------------
# Explained variance & residual diagnostics
# ---------------------------------------------------------------------------

def explained_variance(loadings: np.ndarray, n_features: int) -> Tuple[np.ndarray, np.ndarray]:
    """Return  p e r factor  and cumulative variance shares."""
    var_per_factor = np.sum(loadings ** 2, axis=0) / n_features
    cum = np.cumsum(var_per_factor)
    return var_per_factor, cum


def residual_correlation(returns: pd.DataFrame, loadings: np.ndarray) -> pd.DataFrame:
    """Observed correlation minus reproduced correlation from loadings."""
    reproduced = loadings @ loadings.T
    np.fill_diagonal(reproduced, 1.0)
    resid = returns.corr() - reproduced
    return resid

# Optional quick plots ------------------------------------------------------

def _plot_loadings_heatmap(loadings: np.ndarray, title: str):  # pragma: no cover
    if sns is None or plt is None:
        raise ImportError("seaborn / matplotlib required for plotting.")
    plt.figure(figsize=(12, 9))
    sns.heatmap(loadings, cmap="coolwarm", center=0)
    plt.title(title)
    plt.xlabel("Factors")
    plt.ylabel("Assets")
    plt.tight_layout()
    plt.show()


def _scree_plot(var_per_factor: np.ndarray):  # pragma: no cover
    if plt is None:
        raise ImportError("matplotlib required for plotting.")
    plt.figure(figsize=(8, 5))
    plt.plot(range(1, len(var_per_factor) + 1), var_per_factor * 100, "o-", color="navy")
    plt.xlabel("Factor number")
    plt.ylabel("Explained variance (%)")
    plt.title("Scree plot")
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

B.3 Backtesting

```python
"""alphaefa.backtest     alpha estimation & simple long/short  b a c k tests
==================================================================
This module converts *rotated*  f a c t o r loadings  into

1. **Factor scores**  (\(f_t = (B'B)^{-1}B' x_t\))
2. ** P e r asset  alpha estimates** via OLS regressions
3. **Orthogonal alpha** (alpha component unexplained by the factor space)
4. Quick portfolio constructors ( e q u a l weight  long /  l o n g short )
5. Basic performance metrics (annualised return, volatility, Sharpe)

No plotting is done here     the  h i g h level  notebook can call
`alphaefa.plots.plot_performance()` or your existing Plotly helper.
"""
from __future__ import annotations

from typing import Dict, List, Tuple

import numpy as np
import pandas as pd
import statsmodels.api as sm

__all__ = [
    "compute_factor_scores",
    "estimate_alpha",
    "orthogonal_alpha",
    "select_portfolio",
    "portfolio_returns",
    "performance_metrics",
]

# ---------------------------------------------------------------------------
# Factor scores
# ---------------------------------------------------------------------------

def compute_factor_scores(
    returns: pd.DataFrame,  # T    N
    loadings: np.ndarray,   # N    K (after rotation)
) -> pd.DataFrame:
    """Return factor score matrix (T    K) using regression method."""
    B = loadings
    bt_b_inv = np.linalg.inv(B.T @ B)
    scores = (bt_b_inv @ B.T @ returns.T).T  # T    K
    return pd.DataFrame(scores, index=returns.index,
                        columns=[f"F{i+1}" for i in range(B.shape[1])])

# ---------------------------------------------------------------------------
# Alpha estimation & orthogonal component
# ---------------------------------------------------------------------------
```

```python

def _ols_alpha(y: pd.Series, X: pd.DataFrame) -> Tuple[float, float]:
    """Return (alpha,  p value ) from y ~ c + X OLS."""
    mod = sm.OLS(y, sm.add_constant(X)).fit()
    return float(mod.params[0]), float(mod.pvalues[0])


def estimate_alpha(
    returns: pd.DataFrame,
    factor_scores: pd.DataFrame,
) -> pd.DataFrame:
    """Estimate  p e r asset  alpha &  p value  by regressing returns on factor scores."""
    alphas, pvals = [], []
    for asset in returns.columns:
        a, p = _ols_alpha(returns[asset], factor_scores)
        alphas.append(a)
        pvals.append(p)
    return pd.DataFrame({"alpha": alphas, "p_value": pvals}, index=returns.columns)


def orthogonal_alpha(
    alpha: pd.Series,        # length N
    loadings: np.ndarray,    # N    K
) -> pd.Series:
    """Project alpha onto  c o l u m n space  of loadings & take residual ( orthogonal)."""
    a = alpha.values.reshape(-1, 1)        # N  1
    B = loadings                           # N  K
    lam = np.linalg.inv(B.T @ B) @ B.T @ a # K  1
    alpha_parallel = (B @ lam).ravel()     # N
    return pd.Series(alpha.values - alpha_parallel, index=alpha.index,
                     name="alpha_ortho")

# ---------------------------------------------------------------------------
# Portfolio construction
# ---------------------------------------------------------------------------

def select_portfolio(
    alpha_ortho: pd.Series,
    p_values: pd.Series,
    *,
    n_long: int = 10,
    n_short: int | None = 10,
    p_threshold: float = 0.01,
) -> Tuple[pd.Index, pd.Index]:
    """Return tickers for long and (optionally) short legs using top & bottom alpha."""
    mask = p_values < p_threshold
    alpha_filt = alpha_ortho[mask]
    longs = alpha_filt.nlargest(n_long).index
    shorts = alpha_filt.nsmallest(n_short).index if n_short else pd.Index([])
    return longs, shorts

```

```python
# ---------------------------------------------------------------------------
# Portfolio returns & basic metrics
# ---------------------------------------------------------------------------


def _equal_weight_returns(prices: pd.DataFrame) -> pd.Series:
    r = prices.pct_change().dropna()
    w = pd.Series(1 / prices.shape[1], index=prices.columns)
    return r.mul(w, axis=1).sum(axis=1)


def portfolio_returns(
    prices: pd.DataFrame,
    long_tickers: List[str],
    *,
    short_tickers: List[str] | None = None,
) -> pd.Series:
    """ E q u a l weight  long or  l o n g short  daily simple returns."""
    port_long = _equal_weight_returns(prices[long_tickers])
    if short_tickers:
        port_short = _equal_weight_returns(prices[short_tickers])
        return port_long - port_short  # short leg carries negative weight
    return port_long


def performance_metrics(ret: pd.Series, *, annual_trading_days: int = 252) -> Dict[str, float]:
    """Annualised mean, volatility, Sharpe for a return series."""
    mu = ret.mean() * annual_trading_days
    vol = ret.std(ddof=0) * np.sqrt(annual_trading_days)
    return {
        "ann_return": float(mu),
        "ann_vol": float(vol),
        "sharpe": float(mu / vol) if vol > 0 else np.nan,
    }

def plot_performance(start_date: str,
                     end_date: str,
                     df_long: pd.DataFrame,
                     data: pd.DataFrame,
                     df_short: pd.DataFrame = None):
    # --- 1. Slice & pct_change ---
    tickers_long   = df_long['Ticker'].tolist()
    prices_long    = data[tickers_long].loc[start_date:end_date]
    long_ret       = prices_long.pct_change().dropna()
    w_long         = pd.Series(1/len(tickers_long), index=tickers_long)
    port_long_ret  = long_ret.mul(w_long, axis=1).sum(axis=1)

    # --- 2. Handle shorts if any ---
    if df_short is not None:
        tickers_short   = df_short['Ticker'].tolist()
        prices_short    = data[tickers_short].loc[start_date:end_date]
        short_ret       = prices_short.pct_change().dropna()
        w_short         = pd.Series(-1/len(tickers_short), index=tickers_short)
        port_short_ret  = short_ret.mul(w_short, axis=1).sum(axis=1)
        portfolio_ret   = port_long_ret.add(port_short_ret, fill_value=0)
    else:
```

```
        portfolio_ret = port_long_ret

    # --- 3. Equity curve & rebase ---
    eq_port_raw  = (1 + portfolio_ret).cumprod()
    eq_port      = 100 * eq_port_raw / eq_port_raw.iloc[0]

    # --- 4. Fetch & prep SPX, making end_date inclusive ---
    spx = yf.download('^GSPC',
                      start=start_date,
                      end=(pd.to_datetime(end_date) + pd.Timedelta(days=1)
                          ).strftime('%Y-%m-%d'),
                      auto_adjust=True)['Close']
    spx_ret      = spx.pct_change().dropna()
    eq_spx_raw   = (1 + spx_ret).cumprod()
    eq_spx       = 100 * eq_spx_raw / eq_spx_raw.iloc[0]

      #     after computing eq_port and eq_spx

    # 1) ensure 1 D
    eq_port = eq_port.squeeze()
    eq_spx  = eq_spx.squeeze()

    # 2) align on common dates
    dates = eq_port.index.intersection(eq_spx.index).sort_values()

    # 3) build plot DF via concat

    df_plot = (
    pd.concat([eq_port.loc[dates], eq_spx.loc[dates]], axis=1)
      .set_axis(['Portfolio', 'S&P500'], axis=1)     # no inplace argument
      .reset_index()
      .rename(columns={'index': 'Date'})
    )

    # 4) plot
    fig = px.line(df_plot, x='Date', y=['Portfolio','S&P500'],
                  title=f'Performance␣({start_date}   {end_date})',
                  labels={'value':'Indexed␣to␣100'},
                  template='plotly_white')
    fig.update_layout(
        legend=dict(x=0.02,y=0.95),
        hovermode='x␣unified',
        width=900, height=550
    )

    return fig, portfolio_ret, spx_ret
```